

Bubble Bobble Pygame bug fix X game.py M

26_bubbleBobble Update

Start Over

Go through the entire Bubble Bobble Pygame codebase and identify the existing bug in the bubble trapping logic.

Do not redesign the game or change unrelated functionality.

Find the exact cause of the bug, explain why it happens, and make the smallest code change necessary to fix it.

After making the change, check the code for syntax errors and tell me exactly what you changed.

Optimized tool selection

I'm tracing the bubble-trap logic in the game code now, starting with the exact collision and enemy state handling so I can isolate the root cause and patch only that behavior.

Debugged bubble trapping logic and reviewed 2 files

I've narrowed it to a single comparison in the trap check: it's mixing squared distance with an unsquared radius threshold, which makes trapping only work during nearly perfect overlap. I'm correcting that one condition and then I'll validate the file still parses cleanly.

Updated game.py and ran command

Root cause

In game.py, the bug was in Game.trap_enemies. It used this comparison:

bubble.pos.distance_squared_to(enemy.center) < 24 * 2

game.py

Describe what to build

Agent Auto

Local Default Approvals

Bubble Bobble Pygame bug fix X game.py M

26_bubbleBobble Update

bubble.pos.distance_squared_to(enemy.center) < 24 * 2

distance_squared_to(...) is already measuring a squared distance, so it must be compared against a squared threshold. The code was comparing against 48 instead of 24, which is 576.

That means the trap only triggered for a tiny overlap area, which is why bubbles only caught enemies when they nearly perfectly overlapped. The logic was inconsistent: squared distance vs. unsquared threshold.

Smallest fix

I changed the threshold to a squared value so the collision check matches the geometry:

if bubble.pos.distance_squared_to(enemy.center) < 24 * 24:

This is the only behavior change; the rest of the game logic was left alone.

Syntax check

I verified the file still parses cleanly with:

python -m py_compile game.py

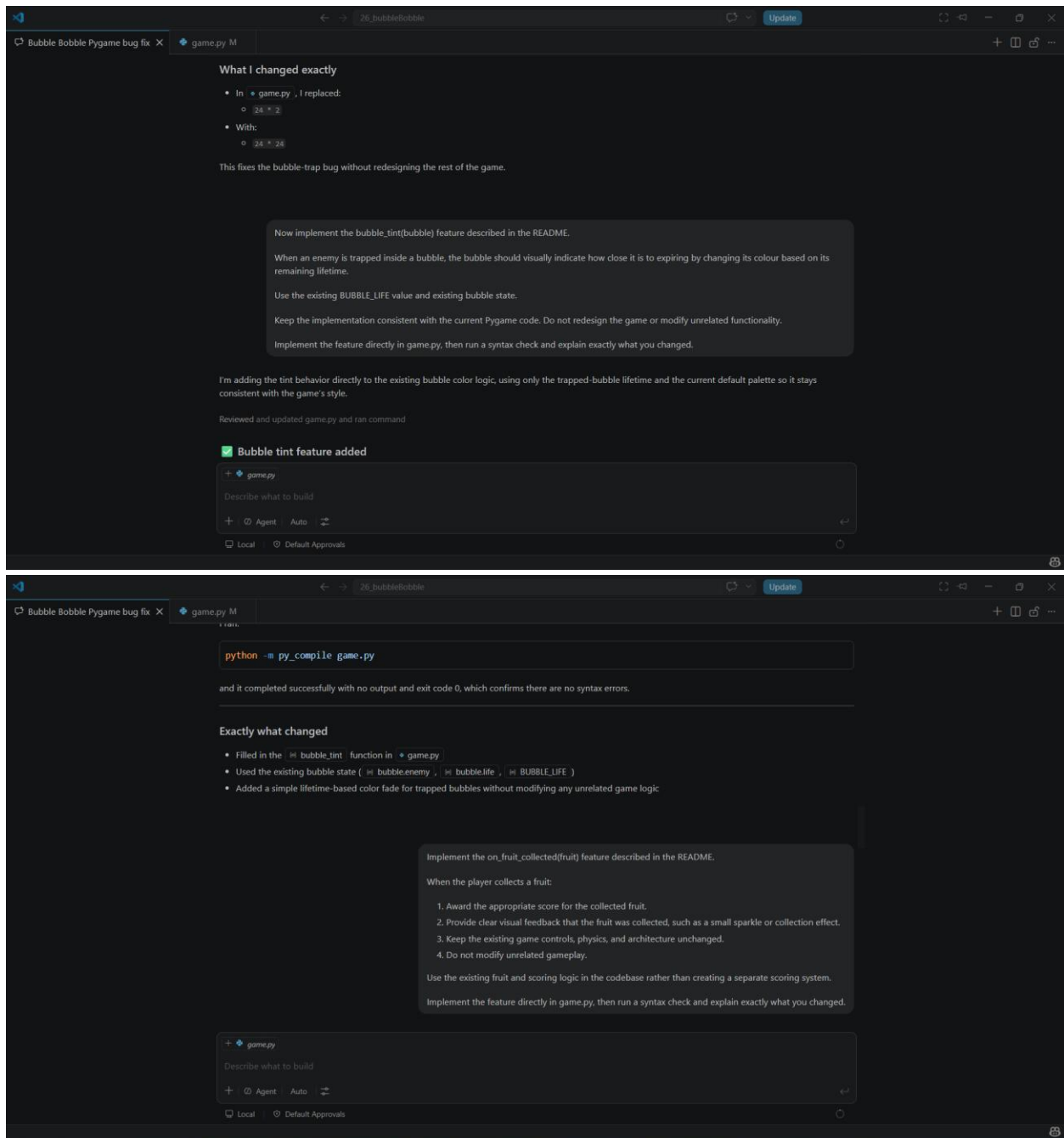
This completed successfully with no output and no errors, confirming there are no Python syntax errors.

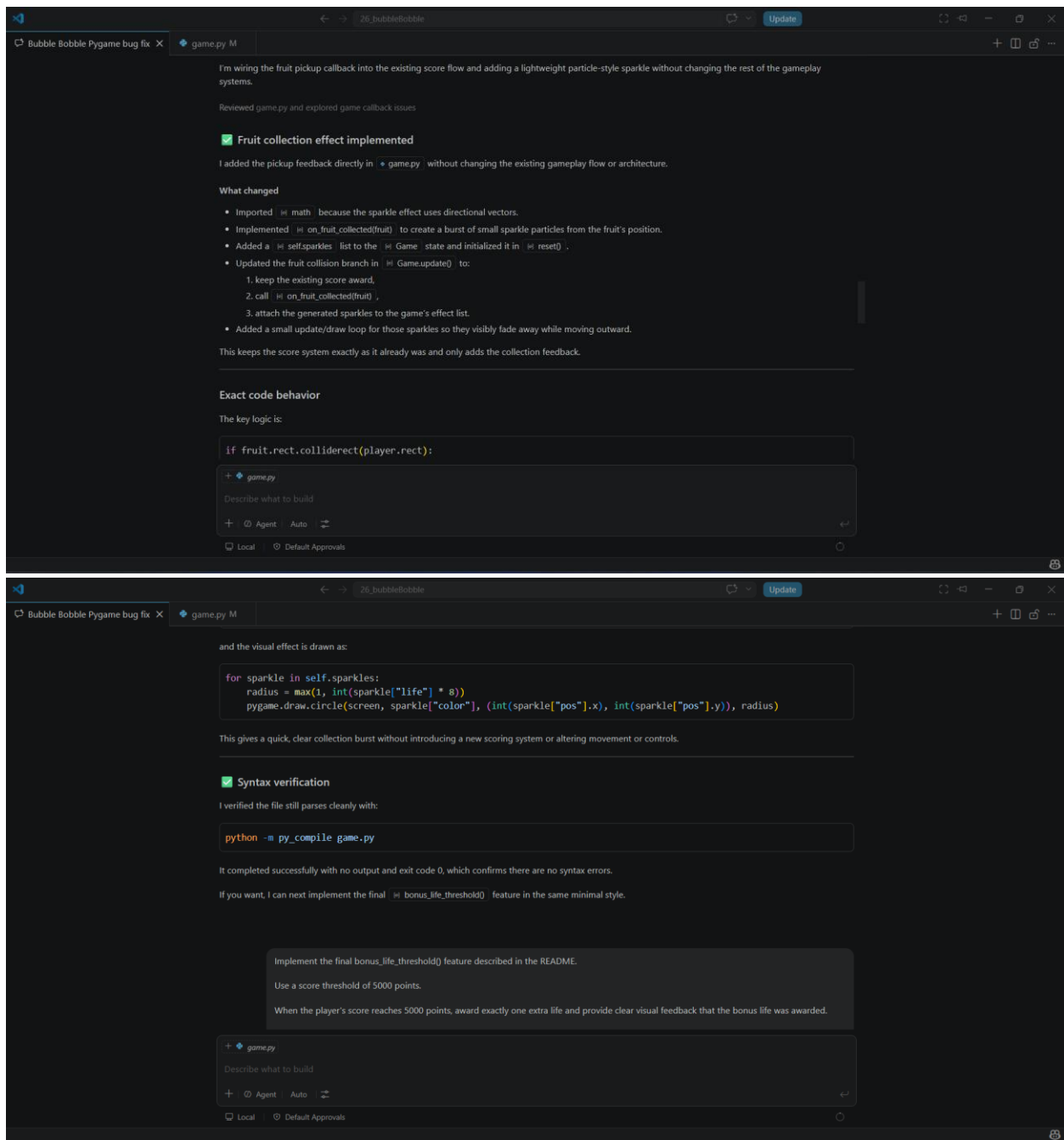
game.py

Describe what to build

Agent Auto

Local Default Approvals





Bubble Bobble Pygame bug fix X

game.py M

26_bubbleBobble

Update

+

+

+

When the player's score reaches 5000 points, award exactly one extra life and provide clear visual feedback that the bonus life was awarded.

Make sure the bonus life is awarded only once for that threshold and is not repeatedly awarded every frame while the score remains above 5000.

Use the existing score and player-life system in the codebase. Do not create a separate scoring or life system.

Keep the existing game controls, physics, graphics, and architecture unchanged.

Implement the feature directly in game.py, then run a syntax check and explain exactly what you changed.

I've got the final feature isolated to the existing score-and-life bookkeeping, and I'm keeping it to a one-time threshold check with a small on-screen reward cue instead of altering the game flow.

Reviewed and updated game.py and ran command `python -m py_compile game.py`

Bonus life feature implemented

I added the final threshold-based life bonus in `game.py` using the existing score and player-life system, with a one-time award and a visible reward indicator.

What I changed

- Implemented `bonus_life_threshold()` to return `5000`
- Kept the existing `self.bonus_awaited` logic in `Game.update()`
- Added a small `self.bonus_text` effect that appears when the award triggers
- Ensured the life is only awarded once per threshold crossing, because the code only increments `self.bonus_awaited` when the score passes a new threshold level

The core logic is:

+

game.py

Describe what to build

+

Agent

Auto

↔

Local Default Approvals

Bubble Bobble Pygame bug fix X

game.py M

26_bubbleBobble

Update

+

+

+

```
self.bonus_awaited = self.score // threshold
self.lives += 1
self.bonus_text = ("+1 LIFE", 0.9)
```

This guarantees:

- at `5000` points, the player gets exactly one extra life
- it does not repeatedly award every frame while staying above `5000`
- it uses the existing life counter, not a separate system

I also added a visible HUD message:

```
if self.bonus_text:
    label = self.font.render(self.bonus_text[0], True, (255, 220, 120))
    screen.blit(label, label.get_rect(center=(WIDTH // 2, 90)))
```

Syntax verification

I ran:

```
python -m py_compile game.py
```

and it completed successfully with no output and exit code 0, confirming there are no syntax errors.

This keeps the controls, physics, graphics, and overall architecture intact while adding the final required bonus-life behavior.

+

game.py

Describe what to build

+

Agent

Auto

↔

Local Default Approvals